

우신희

Game Programmer

현업 1년 7개월 · TypeScript / C# / C++

AI 자동화 & 구조 설계에 강한 게임 프로그래머 전환 준비 중



TypeScript · Phaser · Spine · Canvas 2D · C# · C++ · Unity · Unreal Engine · GAS · Git

WORK EXPERIENCE

현업 경력

교육 콘텐츠 개발사 · 약 1년 7개월

담당 역할

TypeScript 기반 Canvas 2D 콘텐츠 개발

Android WebView 환경
Phaser · Spine · CreateJS · Adobe Animate CC 활용

FPS 21→61 확보 & 로딩 단축 & 반응 레이턴시 감소 등
저사양 기기 기반 최적화 능숙 (Galaxy Tab S6, A)

자동화 & 도구 개발

FTP CLI 배포 스크립트 (PowerShell) Whisper AI 자막 편집기 (80% 시간 단축)

Legacy WebView Version Debug Setting

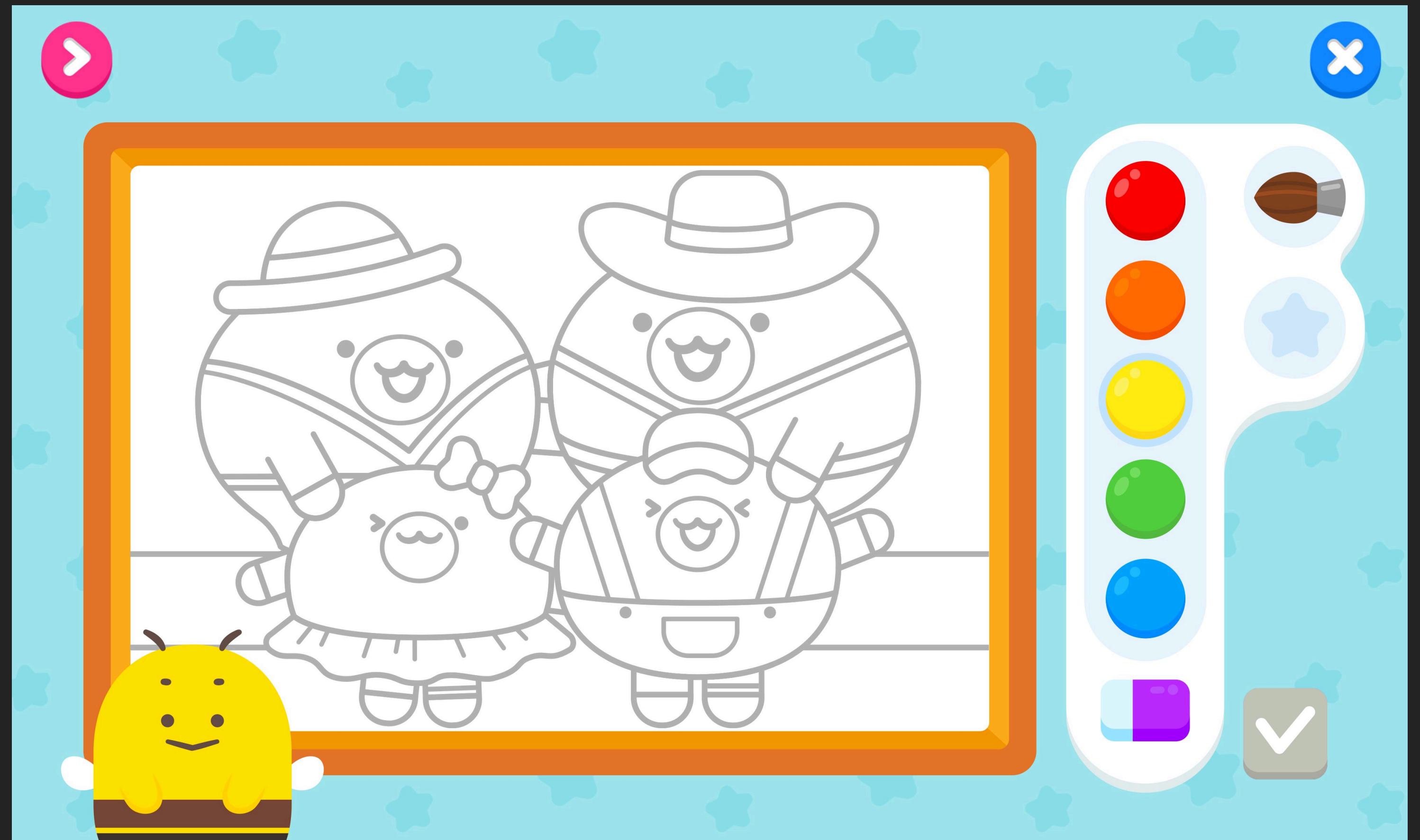
자동 로그인 매크로

우클릭 압축 전송 스크립트

01

아이누리

인터랙티브 교육 콘텐츠
이벤트 드리븐 + MVC 패턴 활용
프레임워크 R&D



유형
교육 콘텐츠

기술
TypeScript
Canvas 2D
Phaser · Spine

역할
콘텐츠 개발

환경
Android WebView

DrawCall 최적화

렌더링 부하를 줄여 **fps 21→61**로 최적화한 사례

시연 영상

<https://youtu.be/cBksAnVZQDg?si=0YtZN22nQZflmIPB>

문제 발견

색칠하기 & 스티커를 찍어 꾸미는 프로젝트에서
무수히 많은 그래픽과 이미지를 찍어내야 했음

색칠하기 또한 하나의 원 그래픽을 만들어 **드래그 이벤트마다 그래픽 객체를 찍어냄**

저사양 타겟 기기인 S6 Lite에서 색칠하기 5초 이상,
스티커 20개 이상 찍힐 때부터 **프레임 드랍 발생**

해결

드래그 이벤트마다 찍어내는 그래픽 객체와 스티커를
찍는 이미지 객체가 객체마다 **DrawCall 호출함**을 인지

Phaser 내장 렌더 텍스처 도입

그래픽 객체와 스티커 객체를 씬에 붙이는 대신,
화면에 그려질 때마다 **렌더 텍스처에 draw**로 구운 뒤
객체를 초기화 하는 방식 사용

즉 무수히 많아질 수 있었던 DrawCall 호출 수를 2회로 고정

실무 적용

색칠하기 & 스티커를 찍을수록
프레임이 드랍되던 현상 수정

FPS: 21



FPS: 61

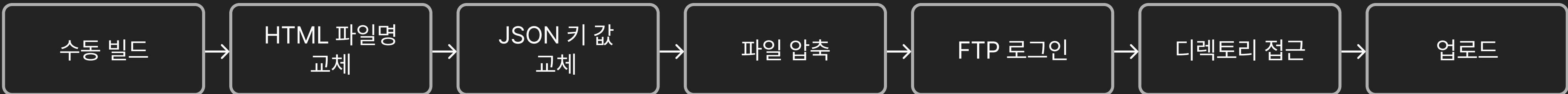
Spine 사용하므로 fps 60 방어가 필수적 | 누적되는 DrawCall 차단

FTP 배포 자동화

빌드 과정에서 병목을 발견하고, CLI 명령어로 배포 자동화

BEFORE

수동 반복



✗ 매 배포마다 위 과정 전체 반복

✗ 휴먼 에러 리스크 + 업무 프로세스 부재

AFTER

CLI 스크립트 체계

.\setup

작업 전 세팅

npm install & branch & commit + push

.\push ""

커밋 & 푸시를 묶어주는 명령어

.\push "아이누리 완료"

.\upload ""

상단 수동 과정을 명령어 한번으로 압축

.env로 보안 정보 분리

OPTIMIZATION CASE

영화 퍼즐 로딩 최적화

외주 맡기던 프로젝트를 재개발하여 로딩 시간 **5초 → 1초**로 단축

시연 영상

<https://youtu.be/s5kyHQpJGSQ>

기존 방식 (외주)

12가지 색 x 모든 영역의 이미지를 전부 로딩

디자이너가 모든 조합을 이미지로 제작해야 함

타겟 기기 Tab A에서 **약 로딩 5초 +**

탭 누적 시 반응 레이턴시 **3-4초 증가**

✓ 장점: 이미지 기반이라 렌더링 정확

✗ 단점: 로딩 느림 + 과도한 디자이너 작업량 + 누적되는 레이턴시

내가 제안한 방식

투명 레이어 1장 + 영역별 고유색 레이어 1장

터치 시 해당 영역의 고유색을 감지 → 선택한 색으로 치환하여 빈 레이어에 칠함

조합 이미지가 필요 없으므로 에셋 로딩 **약 1초로 단축**

반응 레이턴시 **1초 미만**

✓ 장점: 로딩 1~2초 + 디자이너 작업 최소화 + 레이턴시 감소

✗ 단점: 안티앨리어싱 살짝 손실

트레이드 오프 판단: 타겟이 어린이 → 집중력이 짧음 → 빠른 로딩속도를 우선시 함. 안티앨리어싱 손실은 타겟 특성상 체감 미미

Life

How to Choose Healthy and Tasty Summer

In the summer, eating fruit is a healthy way
a guide for picking out two tasty summer f
and blueberries. The watermelon is a juicy
This fruit is 92% water, so it will keep you
A serving of watermelon is also rich in vita
you shop for a watermelon, check the mel



02

뉴스페이퍼 더 지니어스

인터랙티브 교육 콘텐츠

기술

TypeScript · Canvas 2D · CreateJs

역할

콘텐츠 개발

특이사항

Whisper 편집기

Whisper AI 편집기

문제 발견 → 툴 개발 → 적용 → **작업 시간 약 80% 단축 (실측)**

시연 영상

https://youtu.be/M6qNg_z43Ow?si=l-83q1Rp8jq2dvRG

문제

JSON 포맷에 맞게 작업하는 과정에서 **상당한 시간 소모 + 휴먼 에러** 빈발

JSON 데이터만 수정하면 되는 프로젝트인데, 단순 반복 노가다 작업으로 상당한 시간을 소요하는 것은 **팀 전체의 생산성을 떨어뜨린다고 판단**

해결

OpenAI의 Speech To Text 모델 **Whisper AI** 활용 (Base Model)

AI의 API가 끊기는 현상을 막기 위해 AI 모델 다운로드 및 로컬 서버 구동 (Python)

HTML에 Whisper를 탑재하고, 프로젝트의 루트 폴더와 연결하여 **HTML에서 모든 것을 수정할 수 있는 GUI 개발** → 빌드된 프로젝트에서 JSON 수정만으로 양산 완료

80%

작업 시간 단축

AWS Quick 교육 발표 1위

실습 교육 — 약 30명 중



03

다이너 서버이벌

장르

3D 뱀서라이크

엔진 · 언어

Unity · C#

협업 툴

Figma, GitHub

역할

팀장

Game Manager

Player

Level Design

PROJECT 03 — DETAILS

다이너서바이벌 - 상세

다양한 게임 기법을 처음으로 접한 프로젝트

시연 영상

<https://youtu.be/qMQNaMZPN4I?si=XiTgwnWXloAYHjid>

```
// 각 적 스폰에 대해 pool 생성
foreach (var enemyPrefab in enemyPrefabs)
{
    var pool = new ObjectPool<GameObject>(
        createFunc: () =>
        {
            var obj = Instantiate(enemyPrefab.prefab);
            obj.SetActive(false); // 초기 생성 시 비활성화
            return obj;
        },
        actionOnGet: (obj) => InitializeEnemy(obj), // 풀에서 가져올 때 초기화
        actionOnRelease: (obj) =>
        {
            ReleaseEnemy(obj); // 비활성화 시 상태 초기화
            obj.SetActive(false);
        },
        actionOnDestroy: (obj) => Destroy(obj),
        collectionCheck: false,
        defaultCapacity: initialPoolSize,
        maxSize: 100
    );
    pools.Add(enemyPrefab.id, pool);
}
```

스폰 테이블

```
참조 1개
private void SpawnMonstersFromTable(MonsterTableEntry table)
{
    foreach (int monsterId in table.monsterIds)
    {
        enemyPoolManager.SpawnEnemy(monsterId);
    }
}
```

▼ Element 0	Table Id	1
	Trigger Time	1
	Interval	10
▼ Element 1	Table Id	2
	Trigger Time	10
	Interval	30
▼ Element 2	Table Id	3
	Trigger Time	20
	Interval	30
▼ Element 3	Table Id	4
	Trigger Time	60
	Interval	180
▼ Element 4	Table Id	5

배운 점

스폰 테이블을 통한 **Enemy 스폰**

오브젝트 풀링으로 과부하 방지

FSM 상태 패턴으로 적 AI 구현

이 외에도 전체적인 게임 밸런싱 조정 및 TileMap 등
Unity 엔진의 흐름과 **게임 프로그래밍에 대해 학습**

에셋 & 히트박스



PROJECT 03 — TEAM LEAD

다이노 서바이벌 - 팀 리딩

기획부터 시작해 발표까지 팀 리딩

발표 영상

https://youtu.be/0RPa_QnfRfw

인게임 통합 담당

인벤토리 · 적 AI · 스폰 시스템 등

각 파트 인게임 연결

일정 조율 & 커뮤니케이션

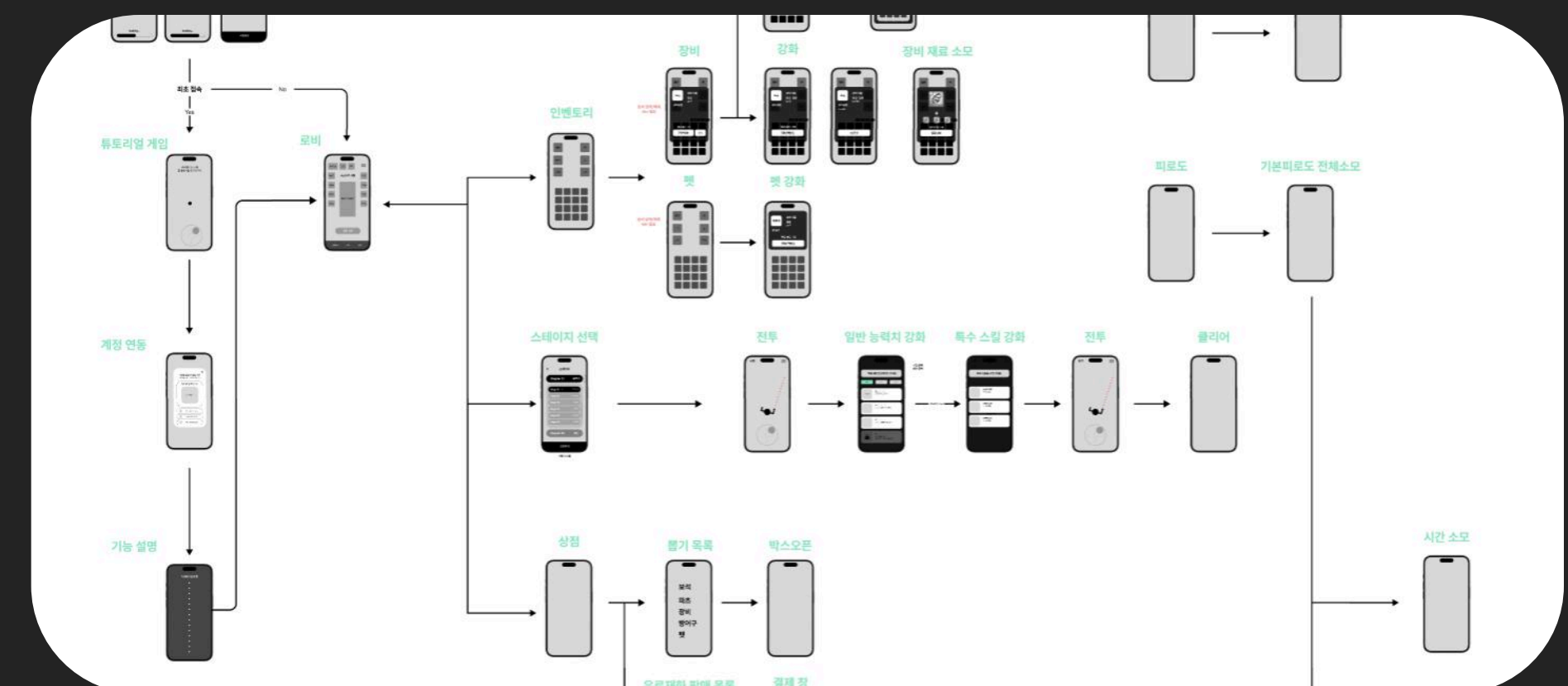
회사 대표 · 매니저와 일정 조율

팀원별 마감 일정 지정 및 진행 상황 관리

최종 발표 직접 진행

게임 기획

Figma를 활용한 UI/레벨 설계 및 게임 기획 문서 작성



CURRENT
LEARNING

Unreal / C++

개인 프로젝트 이후 현재 진행 중인 학습 과정

소울라이크 기반 전투 액션 프로젝트

Unreal Engine 5 · C++ · GAS

GAS 기반 TAG-DRIVEN 입력/무기/콤보 시스템



Unity 경험은 통하지 않았다

소울라이크 기반 전투 액션 프로젝트

시연 영상

<https://youtu.be/pPs-gPu0-0U>



독학 시도

Unity 경험을 앞세워 AI와 함께 독학으로 진행 - 블루프린트, 애님 노티파이 등 기본기를 익히며 Unreal Engine에 적응



회고

언리얼 고유의 설계 구조(GAS, UPROPERTY 활용 등)에 대한 이해 없이 구현 중심으로 진행하여 체력/스태미너, 3단 콤보 공격, 적 AI 추적·공격까지는 구현

기능을 하나씩 추가하다보니 **캐릭터 클래스 하나가 이동 전투 스태미너 사망 처리를 전부 떠맡게 됨**



판단

혼자 부딪혀서 되는 방향이 아니라고 판단

정석 구조 학습을 위해 **GAS(Gameplay Ability System)** 강의 수강

GAS 강의 학습 중

Unreal 엔진의 시스템을 공부하며, 전체적인 플로우 이해

시연 영상

<https://youtu.be/w0JdObCyL9g>

GAS Warrior 프로젝트

Unreal Engine 5 + GAS 기반 액션 게임

GA - 플레이어의 행동

GE - 플레이어의 상태 이상(Infinite, duration, Instant)

Attribute - 플레이어의 데이터

Gameplay Tag - 플레이어의 행동 상태

플레이어의 능력을 추가할 때, C++ 코드 추가 대신
태그 추가 + BP 연결로 구현 가능 → 확장성 증가

```
void UWarriorAbilitySystemComponent::GrantHeroWeaponAbilities(const TArray<FWarriorHeroAbilitySet>
{
    if (InDefaultWeaponAbilities.IsEmpty())
    {
        return;
    }

    for (const FWarriorHeroAbilitySet& AbilitySet : InDefaultWeaponAbilities)
    {
        if (!AbilitySet.IsValid()) continue;
        FGameplayAbilitySpec AbilitySpec(TSubclassOf<UGameplayAbility>(AbilitySet.AbilityToGrant));
        AbilitySpec.SourceObject = GetAvatarActor();
        AbilitySpec.Level = ApplyLevel;;
        AbilitySpec.DynamicAbilityTags.AddTag(AbilitySet.InputTag);

        OutGrantedAbilitySpecHandles.AddUnique(GiveAbility(AbilitySpec));
    }
}
```

▼ 태그	
AssetTags (Default AbilityTags)	Player.Ability.Attack.Light.Axe X ▼
태그 포함 어빌리티 취소	비어 있음 ▼
태그 있는 능력 차단	Player.Ability.Attack X ▼ Player.Ability.Equip X Player.Ability.Unequip X
활성화 소유 태그	비어 있음 ▼

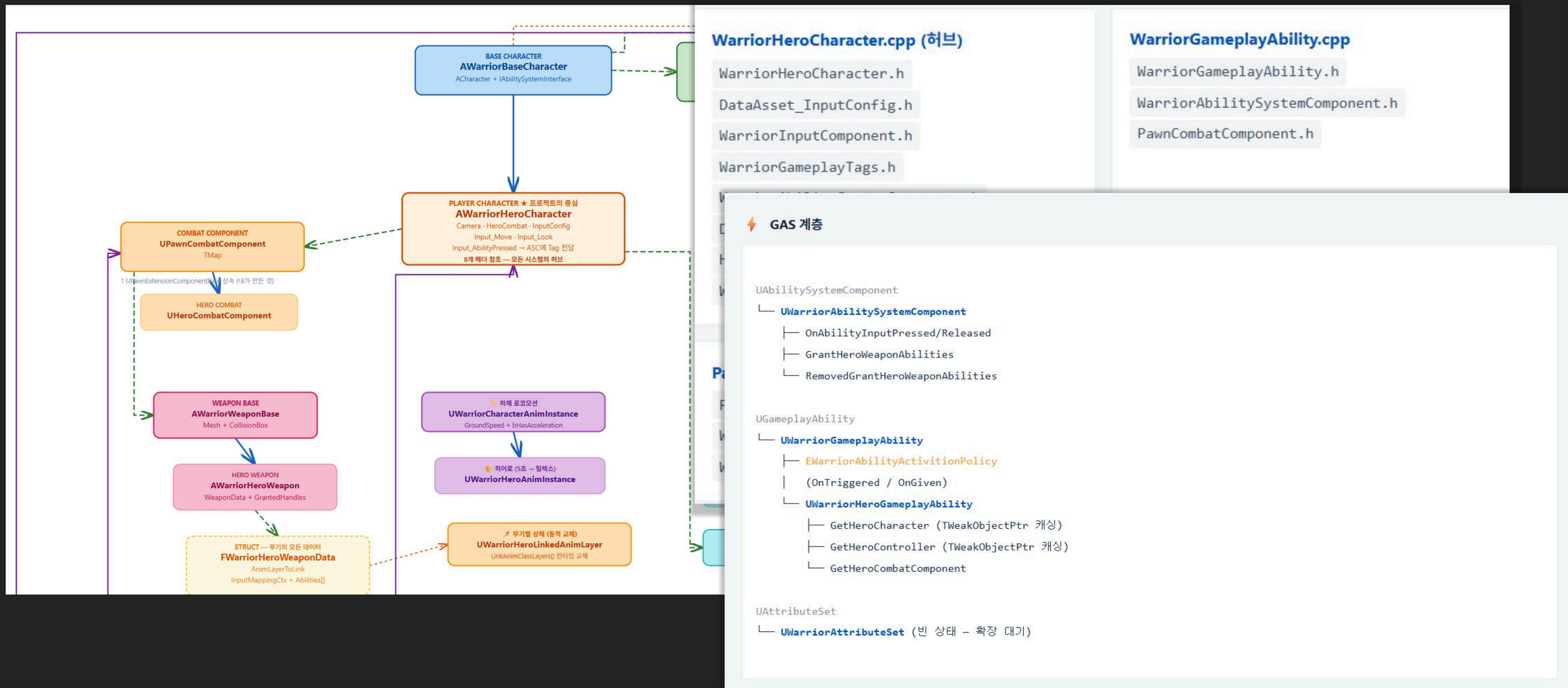
CURRENT LEARNING — REPEAT

강의를 타이핑만 하는가?

AI로 대시보드를 설계 · 정리하여 구조 파악 + cs 개념 정리

링크

<https://wooshinhui.github.io/cs-dashboard/>



LEARNING ROADMAP

학습 로드맵

PAST



Unity 게임 개발

다이노 서바이벌 (팀장)
게임 프로그래밍 기초 습득
오브젝트 풀링 / FSM
히트박스 / AI
데이터 파싱 / 스폰 테이블

CURRENT



CS 기초 + C++

자료구조 / 알고리즘
운영체제 / 네트워크
포인터 · 스마트 포인터
컴퓨터 구조

NOW →



GAS Warrior 프로젝트

GAS Warrior 프로젝트
GA / GE / Attribute
Gameplay Tag
대시보드 문서화

병목을 해결하는 프로그래머

오늘의 병목은 무엇이셨나요?

우신희 | Game Programmer



<https://github.com/WooShinHui/Warrior>



ewew3069@gmail.com